Act as a multidisciplinary principal review team: system architect, cyber-security architect, senior frontend engineer, UI/UX specialist, product strategist, business logic analyst, and SaaS growth advisor.

Project

Stoquify / AqStoqFlow enterprise management platform.

Workspace

`E:\ohada saas\Focused projects\stoquify`

Domain

Platform-wide remediation, source-of-truth convergence, financial integrity, tenant isolation, security hardening, operational resilience, and enterprise release assurance.

Mission

Using the completed platform workflow audit as the authoritative remediation input, design and execute a professional, modern, dependency-ordered hardening program that corrects every confirmed or evidence-supported deficiency without destabilizing the platform.

The program must move Stoquify toward becoming a secure and authoritative tenant-level source of operational, financial, inventory, commercial, workforce, payroll, compliance, approval, audit, and control truth.

Do not attempt a monolithic rewrite. Convert the audit findings into bounded remediation workstreams with explicit ownership, dependencies, migrations, tests, rollout controls, rollback procedures, and executable acceptance gates.

Address findings in priority order:

1. P0 tenant-isolation and financial-integrity defects.
2. P1 security, entitlement, provider-truth, concurrency, audit, statutory, and migration defects.
3. P2 consistency, resilience, observability, duplication, assurance-coverage, and productivity weaknesses.
4. P3 enhancements only after the authoritative foundations are complete.

A finding is not remediated merely because code exists. Completion requires passing tests, migration evidence where applicable, behavior verification, rollback readiness, and updated source-of-truth documentation.

Authoritative Input Artifacts

Read these artifacts before planning or changing code:

- `what-next/workflow-assurance/STOQUIFY_PLATFORM_WORKFLOW_SYSTEM_AUDIT_2026-07-28.md`
- `what-next/workflow-assurance/STOQUIFY_PLATFORM_WORKFLOW_REGISTRY_2026-07-28.json`

- `what-next/workflow-assurance/STOQUIFY_PLATFORM_SOURCE_OF_TRUTH_MAP_2026-07-28.md`
- `what-next/workflow-assurance/STOQUIFY_PLATFORM_WORKFLOW_REMEDIATION_ROADMAP_2026-07-28.md`
- `what-next/workflow-assurance/STOQUIFY_PLATFORM_WORKFLOW_INCONSISTENCY_REGISTER_2026-07-28.json`
- `docs/system-audit/STOQUIFY_PLATFORM_WORKFLOW_SYSTEM_AUDIT_2026-07-28.pdf`
- `AGENTS.md`
- current `what-next/` readiness, blocker, release, and policy-gate artifacts
- `graphify-out/` architecture reports and graphs

Cross-check every audit finding against the current source before implementing it. The repository is active, and some findings may already be partially addressed.

Record each finding as:

- Still present.
- Partially remediated.
- Fully remediated and verified.
- Superseded by an approved architectural decision.
- Blocked by external input.
- Invalidated by newer evidence.

Never silently discard a finding.

Principal Domain Lenses

Act as a senior enterprise remediation and hardening team:

- Enterprise software architect: preserve domain boundaries, service ownership, dependency direction, compatibility contracts, and incremental delivery.
- Cybersecurity architect: enforce tenant isolation, least privilege, RBAC, module entitlement, trusted-origin policy, fresh authentication, secrets hygiene, redaction, abuse resistance, and auditable security decisions.
- Financial integrity architect: protect payment, store-credit, inventory, receivable, cash-drawer, AP, payroll, ledger, reconciliation, correction, and close truth.
- Data architecture specialist: identify authoritative records, eliminate conflicting sources of truth, preserve provenance, and control projection freshness.
- Reliability engineer: implement idempotency, concurrency protection, retries, durable queues, observability, recovery procedures, and rollback evidence.
- UI/UX workflow specialist: expose authoritative state, blockers, next actions, freshness, provisional states, corrections, and recovery without allowing the UI to own business truth.
- OHADA/SYSCOHADA-aware platform architect: keep statutory, tax, accounting, fiscal, and country-pack rules configurable and versioned, with expert-approved provenance.
- SaaS modularity specialist: preserve tenant-safe module provisioning, entitlement enforcement, scalability, operability, and commercialization boundaries.
- Test and release assurance specialist: require adversarial, integration, migration, regression, and release-gate evidence before closing findings.

Lead Orchestration Model

Use one lead program orchestrator. Assign bounded, non-overlapping workstreams to specialist agents. Do not let agents independently redesign shared contracts.

Lead Program Orchestrator

Responsibilities:

- Freeze the current evidence baseline.
- Maintain the remediation register.
- Define workstream dependencies.
- Assign exclusive ownership of files and contracts.
- Prevent overlapping edits.
- Reconcile shared schema, event, permission, and read-model changes.
- Enforce phase-entry and phase-exit gates.
- Stop downstream implementation when an upstream truth boundary is unresolved.
- Produce the consolidated progress report and final release recommendation.

Specialist Workstreams

1. Tenant and Access-Boundary Agent

Own:

- Organization-scoped database access.
- Prisma client or approved RLS/repository alternative.
- Raw-client escape policy.
- Cross-tenant adversarial tests.
- RBAC and module-entitlement enforcement.
- Action, API, worker, and service authorization matrices.

Do not modify payment or accounting business semantics.

2. Security and Identity Hardening Agent

Own:

- Static trusted-origin policy.
- Proxy-header normalization.
- Transactional critical-audit outbox.
- Invitation-token hashing and one-time redemption.
- Fresh-auth and step-up boundaries.
- Sensitive-data redaction.
- Security logging and abuse controls.

3. Purchasing and Approval-Control Agent

Own:

- Purchase-order approval state machine.
- Removal of generic bulk-approval bypass.
- Maker-checker and segregation-of-duties rules.
- Append-only approval transitions.
- AP reconciliation exception idempotency.
- Supplier-payment state vocabulary.

4. POS and Payment-Truth Agent

Own:

- Store-credit containment and immutable ledger.
- Provider-authoritative tender states.
- Refund execution and settlement evidence.
- Customer receivable concurrency.
- Cash-drawer concurrency.
- Partial return and correction lineage.
- Payment-to-ledger and payment-to-reconciliation integration.

5. Inventory Integrity Agent

Own:

- Inventory event idempotency ordering.
- Duplicate blocker-evidence prevention.
- Atomic production consumption/output.
- Movement, valuation, reversal, and close-invalidation correctness.
- Concurrent stock mutation tests.

6. Reconciliation and Accounting Agent

Own:

- `Payment` to `PaymentTransaction` transition policy.
- Provider event, statement, match, suspense, ledger, and close relationships.
- Cash, receivable, refund, settlement, and as-of semantic contracts.
- Accounting source links.
- Certified close/export blockers.

7. HRIS, Payroll, and Compliance Agent

Own:

- Payroll evidence and correction lifecycle.
- Provider settlement evidence.
- Country-pack provenance.

- Statutory source hashes and effective dates.
- Expert-approval requirements.
- Close integration and redaction.

Do not certify legal or statutory correctness. External expert validation remains mandatory.

8. Read-Model and UI Consistency Agent

Begin only after authoritative financial semantics are approved.

Own:

- Dashboard, finance, snapshot, report, and analytics convergence.
- Freshness and provenance indicators.
- Provisional versus settled states.
- Role-aware recovery and corrective actions.
- Duplicate route and action retirement.
- Responsive behavior and accessibility verification.

9. Reliability, Observability, and Release Agent

Own:

- External logs, metrics, alerts, traces, redaction, and retention.
- Delivery tests and paging drills.
- Runtime assurance workers.
- Migration-history health.
- Release-secret readiness.
- Rollback and recovery runbooks.
- Evidence-bound release gates.

Skill Routing

Use installed skills only after reading each relevant `SKILL.md`. Prefer the smallest applicable set.

Suggested routing:

- Program coordination:
  - `001-aqstoqflow-program-orchestrator`
  - `002-aqstoqflow-control-plane`
  - `aqstoqflow-gap-plan-orchestrator`
- Tenant, security, and modules:
  - `aqstoqflow-access-boundary-hardener`
  - `aqstoqflow-module-access-guard-contract`
  - `aqstoqflow-module-entitlement-schema`
  - `aqstoqflow-module-security-prerequisites`

    - `aqstoqflow-tenant-entitlement-provisioning`
- POS, inventory, payment, and AP:
    - `007-aqstoqflow-pos-ledger-controls`
    - `014-aqstoqflow-offline-pos-sync`
    - `aqstoqflow-offline-pos-replay-finalizer`
    - `010-aqstoqflow-inventory-boundary-gate`
    - `010-aqstoqflow-inventory-valuation-kernel`
    - `009-aqstoqflow-payment-reconciliation-moat`
    - `011-aqstoqflow-purchasing-ap-controls`
- Payroll, compliance, and close:
    - `aqstoqflow-payroll-kernel-hardener`
    - `aqstoqflow-payroll-payment-recon`
    - `aqstoqflow-payroll-accounting-close`
    - `006-aqstoqflow-country-pack-factory`
    - `008-aqstoqflow-compliance-center`
    - `023-aqstoqflow-close-assurance-suite`
- AI, assurance, and release:
    - `016-aqstoqflow-ai-copilot-guardrails`
    - `017-aqstoqflow-enterprise-release-gate`
    - `aqstoqflow-release-verification-foundation`
    - `003-aqstoqflow-error-notification-foundation`

Do not invoke every skill automatically. The orchestrator must justify the selected skill for each workstream and prevent duplicated responsibility.

Universal Engineering Principles

- Begin with current evidence, not the previous report alone.
- Preserve the dirty worktree and unrelated user changes.
- Make surgical, dependency-aware changes.
- Keep mutations service-owned and server-side.
- Treat dashboards and AI outputs as projections, never authorities.
- Preserve append-only evidence, reversal history, corrections, source links, and audit lineage.
- Never weaken a passing control to simplify implementation.
- Every tenant-owned read and mutation must fail closed outside the active organization.
- Every externally callable mutation must explicitly enforce permission and module entitlement.
- Use fresh authentication and maker-checker controls where risk warrants them.
- Do not transform provisional provider evidence into settled financial truth.
- Do not hard-code country-specific statutory conclusions.
- Do not rewrite all affected domains simultaneously.

- Use compatibility adapters, feature flags, reconciliation reports, and staged migrations.
- Do not perform destructive Prisma operations.
- Do not modify production data.
- Do not expose secrets, personal data, payroll data, provider payloads, fiscal identifiers, or payment details in reports.
- Stop for authorization before an externally visible, destructive, or materially scope-expanding action.

Phase 0 — Baseline and Immediate Containment

Objectives

Prevent additional exposure while the structural fixes are built.

Tasks

1. Revalidate all 20 inconsistency-register findings against current source.
2. Produce a file-level and model-level impact map.
3. Disable store-credit tender unless an authoritative store-credit instrument and sufficient balance can be proven.
4. Remove `APPROVED` from generic purchase-order bulk status mutation.
5. Prevent provisional electronic tenders and non-cash refunds from being represented as settled.
6. Reject unapproved Host and forwarded-host origins.
7. Preserve current statutory and certified-export blockers.
8. Introduce no schema migration during containment unless required for safe denial.

Exit Criteria

- Store-credit attempts fail closed without posting financial effects.
- Every PO approval uses the canonical approval command.
- Provisional payments and refunds cannot appear as settled.
- Spoofed-origin tests fail closed.
- Existing happy paths remain operational.
- Containment can be rolled back without removing audit evidence.

Phase 1 — Tenant, Security, and Authorization Boundaries

Objectives

Create fail-closed organization isolation and durable security evidence.

Tasks

1. Decide and document whether ADR-0002 will be implemented or superseded.
2. Implement one approved organization-scoping architecture:
    - Scoped Prisma extension; or
    - Tenant-scoped repositories plus a prohibited raw client; or
    - PostgreSQL RLS combined with controlled infrastructure access.
3. Provide an explicit, reviewed unscoped escape for migrations and infrastructure only.
4. Add a static gate that rejects unauthorized raw-client use.
5. Test nested identifiers and indirect entity IDs across tenants.
6. Make module entitlement mandatory for externally callable mutations.
7. Replace request-derived trusted origins with a static normalized allowlist.
8. Add transactional append-only security audit obligations.
9. Store invitation-token digests only.
10. Redeem invitations using one conditional transition.
11. Persist offline event actor identity and introduce narrow worker/manual replay authority.

Exit Criteria

- Cross-tenant adversarial tests pass for representative domains.
- Raw-client gate passes with no unexplained escape.
- Page/action/API/worker/service permission and entitlement matrices reconcile.
- Audit-database failure cannot silently erase a critical decision obligation.
- Concurrent invitation redemption produces one winner.
- Offline replay preserves the original actor and cannot escalate authority.

Phase 2 — Financial and Operational Truth

Objectives

Make payment, store-credit, cash, receivable, inventory, AP, refund, and reconciliation outcomes authoritative and concurrency-safe.

Tasks

1. Create a tenant/customer-scoped immutable store-credit ledger.
2. Implement reserve, consume, expire, reverse, and reconcile commands.
3. Introduce explicit payment states:
    - Provisional.
    - Authorized.
    - Captured.
    - Provider accepted.
    - Settled.

    - Failed.
    - Reversed.

4. Require signed or independently verified provider evidence before settlement.
5. Post provisional value through clearing or suspense accounts.
6. Implement provider refund request, acceptance, settlement, failure, retry, and reconciliation.
7. Replace customer and drawer read-compute-set updates with CAS, atomic increments, or immutable-ledger projections.
8. Correct inventory idempotency ordering and duplicate blocker evidence.
9. Make AP exception creation deterministic and concurrency-safe.
10. Publish and enforce the legacy `Payment` to `PaymentTransaction` migration/read policy.
11. Add partial-return support only after the payment, tax, stock-cost, and ledger-allocation contracts are authoritative.

Exit Criteria

- Concurrent sales cannot exceed customer credit.
- Concurrent drawer operations preserve every transaction.
- Store credit cannot be forged, overspent, or consumed twice.
- Duplicate provider messages produce one financial effect.
- Provider refunds remain pending until authoritative evidence exists.
- Payment, reconciliation, ledger, and statement totals tie out.
- Inventory replay produces no duplicate movement or blocker evidence.
- Corrections preserve complete reversal lineage.

Phase 3 — Projection and Workflow Convergence

Objectives

Ensure every role and interface receives a consistent projection of the same authoritative truth.

Tasks

1. Establish a versioned semantic catalog for:
    - Cash collected.
    - Receivables.
    - Partial payments.
    - Refunds.
    - Provider settlement.
    - Store credit.
    - Supplier release versus settlement.
    - Tax.

    - Time zone and as-of date.
2. Build shared golden fixtures and reconciliation assertions.
3. Migrate dashboard, finance, tenant snapshot, daily reports, close reports, and analytics to shared adapters.
4. Require every derived surface to expose:
    - Source.
    - Semantic version.
    - As-of time.
    - Freshness.
    - Reconciliation status.
    - Source hash.
    - Blockers.
    - Redaction state.
5. Consolidate duplicate actions and routes using tested adapters, redirects, and deprecation markers.
6. Add role-aware next actions and recovery paths.
7. Verify keyboard access, screen-reader semantics, focus management, responsive layouts, safe errors, and localization.

Exit Criteria

- The same fixture produces identical totals across all relevant surfaces.
- Provisional, settled, blocked, stale, partial, and corrected states are never conflated.
- No dashboard recalculates financial truth using private status filters.
- Duplicate routes either converge on one service/read model or are retired safely.
- Browser smoke and accessibility evidence cover representative roles.

Phase 4 — Assurance, Observability, and Enterprise Release

Objectives

Make the hardened platform observable, recoverable, evidence-bound, and ready for controlled release evaluation.

Tasks

1. Expand workflow assurance coverage to:
    - Authentication and invitation.
    - Tenant isolation.
    - RBAC and entitlement.
    - HRIS.
    - Master data.

    - Evidence retention/redaction.
    - Public APIs and integrations.

2. Resolve production migration-history blockers without reset or history rewriting.
3. Provision independent managed release secrets.
4. Wire production-grade logs, metrics, traces, alerts, and redaction.
5. Test alert delivery, paging, retention, escalation, replay, and operator recovery.
6. Add provider/webhook signature, timestamp, idempotency, replay, and quarantine controls where integrations exist.
7. Bind country packs to signed/versioned source artifacts, effective dates, expert approval, and expiry/revalidation.
8. Run backup, restore, rollback, queue replay, provider outage, and partial-deployment drills.
9. Bind release evidence to a clean candidate commit.
10. Keep final release authority outside the implementing agents.

Exit Criteria

- Every release blocker is closed with executable evidence or explicitly remains blocking.
- Migration history is healthy on the approved deployment target.
- Observability transports and paging drills are independently verified.
- Provider/webhook boundaries pass authenticity and replay tests.
- Statutory packs have expert-reviewed provenance.
- Rollback preserves financial and audit history.
- No agent self-certifies security, accounting, statutory compliance, or production readiness.

Finding Completion Contract

For each inconsistency-register item, maintain:

- Finding ID.
- Original priority.
- Current evidence.
- Root cause.
- Containment.
- Permanent correction.
- Authoritative owner.
- Files and models affected.
- Schema and migration impact.
- Security and privacy impact.
- Financial and accounting impact.
- UI and workflow impact.
- Tests added.
- Commands executed.
- Rollout state.

- Rollback procedure.
- Residual risk.
- Independent reviewer.
- Final status.

Allowed final statuses:

- Verified remediated.
- Partially remediated.
- Blocked external.
- Deferred with accepted risk.
- Superseded by approved decision.

“Implemented” is not an accepted final status.

Required Artifacts

Save program artifacts under:

`what-next/platform-hardening/`

Produce:

1. `STOQUIFY_PLATFORM_HARDENING_MASTER_PLAN_2026-07-28.md`
2. `STOQUIFY_PLATFORM_REMEDIATION_REGISTER_2026-07-28.json`
3. `STOQUIFY_PLATFORM_DEPENDENCY_AND_OWNERSHIP_MAP_2026-07-28.md`
4. `STOQUIFY_PLATFORM_SECURITY_HARDENING_PLAN_2026-07-28.md`
5. `STOQUIFY_PLATFORM_FINANCIAL_TRUTH_MIGRATION_PLAN_2026-07-28.md`
6. `STOQUIFY_PLATFORM_READ_MODEL_CONVERGENCE_PLAN_2026-07-28.md`
7. `STOQUIFY_PLATFORM_TEST_AND_ASSURANCE_MATRIX_2026-07-28.json`
8. `STOQUIFY_PLATFORM_ROLLOUT_AND_ROLLBACK_PLAN_2026-07-28.md`
9. `STOQUIFY_PLATFORM_HARDENING_EXECUTION_LEDGER_2026-07-28.md`
10. `STOQUIFY_PLATFORM_HARDENING_FINAL_READINESS_2026-07-28.md`

For each implementation slice, update the register and execution ledger rather than creating disconnected reports.

Verification Strategy

Inspect `package.json` before selecting commands. Run only checks relevant to the current workstream.

Baseline candidates:

```
npm run prisma:validate
npm run typecheck
npm run service:boundary:fail
npm run hard-delete:fail
npm run workflow:assurance:release-gate
```